MisInfo AI

A plain-English guide to MisInfo AI v2: what every part of the code does, how Docker runs it on your Mac, how the website reaches its verdicts, and how to change things with confidence.

BREAKING!!! Drinking hot lemon water every morning kills cancer cells. Share this before they delete it!

Mount Everest is the highest mountain above sea level.

Prince Kofi Amoah · github.com/pakson556/misinfo-ai · branch v2

October 2026

Contents

Read sections 1–4 first: they explain the whole system and how to run it. The rest is reference for when you want to change something.

1 The big picture

2 What happens when you press “Check it”

3 Docker, explained from zero

4 Running the app (four ways)

5 Using the website and reading a result

6 How the verdict is calculated

7 Settings and API keys

8 Backend code tour, file by file

9 Frontend code tour, file by file

10 Training the style model

11 Tests, CI and Git

12 Troubleshooting

13 What changed from v1, and why

14 Glossary

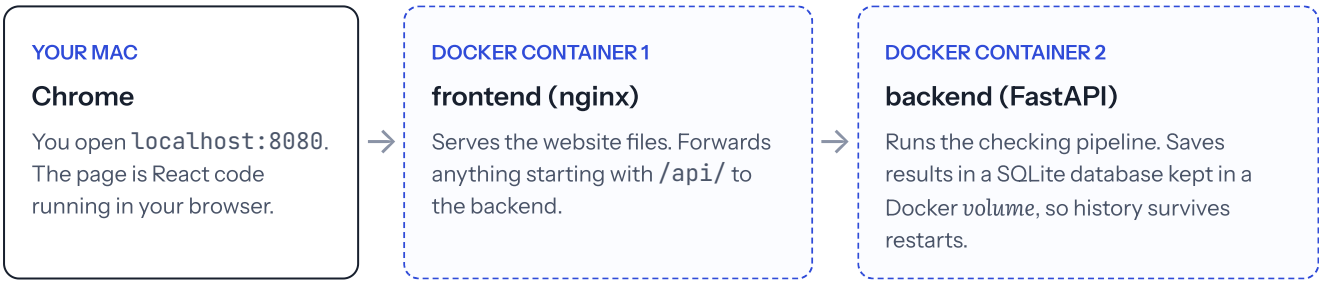
1 The big picture

MisInfo AI checks whether content is true. You paste text, a link, or a screenshot. It finds each factual claim, looks for evidence, and shows you which parts hold up, with every source linked.

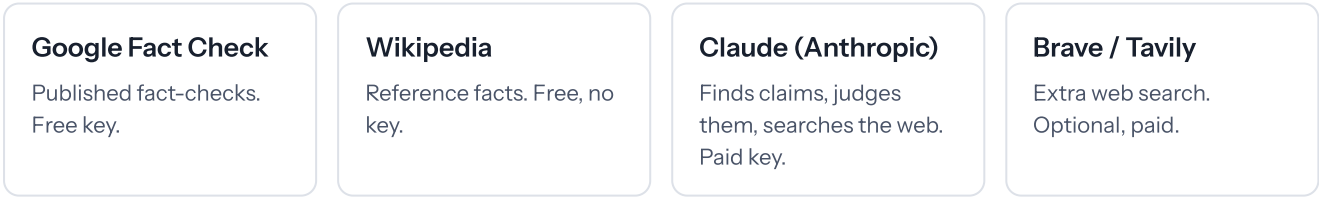
The project has three parts, each in its own folder:

Folder	What it is	Built with
`backend/`	The **API**: the "brain". Receives text, finds claims, gathers evidence, asks the AI to judge, combines the answers, and saves history.	Python, FastAPI
`frontend/`	The **website** you see in the browser. Sends your input to the backend and draws the results live as they arrive.	React, Tailwind CSS, Vite
`training/`	Scripts to **train an optional ML model** on your datasets (LIAR, PHEME, FakeNewsNet…). The app runs fine without it.	scikit-learn, PyTorch

How the pieces connect



The backend then calls out to these services over the internet, all optional:



The one idea that makes v2 different

Evidence decides the verdict, not writing style. Version 1 tried to guess whether text *looked* like misinformation. That's unreliable: true news can be dramatic and lies can be calm. Version 2 checks each claim against real sources. Writing style is still analysed and shown (the dotted underlines), but it can only nudge the score when there's no evidence, and confidence stays low when evidence is missing.

Basic mode vs full mode

With **no API keys** the app still works: it finds claims with simple rules, spots persuasion tactics, and matches published fact-checks. Most claims then come back `Unverified`, which is honest rather than broken. Adding the **Google Fact Check** key (free) and an **Anthropic** key (paid) turns on real verdicts. Section 7 covers keys.

2 What happens when you press “Check it”

Here is one check followed from start to finish. The file that does each step is shown in blue, so you know where to look in the code.

1. **The browser sends your text**

   The website makes a `POST` request to `/api/analyze/stream` with `{"text": "...", "use_web": true}`. For a screenshot it uploads the image to `/api/analyze/image/stream` instead.

   `frontend/src/lib/api.js`

2. **nginx forwards it to the backend**

   The frontend container sees the `/api/` prefix and passes the request to the backend container.

   `frontend/nginx.conf`

3. **The backend checks and prepares the input**

   FastAPI validates the request (text *or* link, not too long, rate limit not exceeded). A link is downloaded and the article text extracted. A screenshot is read with OCR (Tesseract).

   `backend/app/main.py` · `pipeline/ingest.py`

4. **Find the claims**

   The AI lists up to 5 checkable factual claims, skipping opinions and jokes, and rewrites each to make sense on its own. It also returns the exact quote each claim came from, so the page can highlight it. With no AI, simple sentence rules pick factual-looking sentences instead.

   `pipeline/orchestrator.py` → `llm/`

5. **Analyse the wording (instant, no internet)**

   Regular expressions look for persuasion tactics: pressure to share, urgency, conspiracy framing, fake authority, miracle cures, ALL CAPS, “!!!”. If a trained style model is installed, it scores the text too.

   `pipeline/tactics.py` · `pipeline/style.py`

6. **Check every claim at the same time**

   For each claim, in parallel: search Google Fact Check and Wikipedia (plus Brave/Tavily if configured), then give the claim and the evidence to the AI, which returns a verdict, a confidence and a plain-English reason. Claude may also run its own web searches. If the AI fails, published fact-checks are matched instead.

   `pipeline/retrieval.py` · `llm/anthropic_provider.py`

7. **Combine into one verdict**

   Claim verdicts, weighted by importance and confidence, become an overall verdict, a risk score and a confidence score (section 6 has the maths). The AI writes a headline and a 2–3 sentence summary.

   `pipeline/aggregate.py`

8. **Save and finish**

   The full result is saved to the database for the History and Insights pages, and sent to the browser.

   `backend/app/db.py`

Why the page updates live

The backend doesn't wait until the end to reply. It keeps the connection open and sends small **events** as it works, using a standard called **Server-Sent Events (SSE)**. That's why claims appear grey and then turn red or green one by one.

Event	Sent when	What the page does
`input`	Immediately	Shows your text (or the text read from a link or screenshot)
`stage`	Each step starts or ends	Updates the 1–2–3 progress steps
`claims`	Claims found	Highlights them in grey (“checking”)
`tactics` , `style`	Wording analysed	Adds dotted underlines and the “How it's written” panel
`claim_result`	Each claim finishes	Colours that claim and shows its sources
`result`	All done	Shows the verdict, headline, risk bar and guidance
`error`	Something failed	Shows a red error message

> **It is built not to crash**
>
> Every outside call can fail without breaking the check. If Wikipedia is down you get a warning, not an error. If the AI fails, the app falls back to fact-check matching. If a claim takes too long (75-second budget), it's marked unverifiable. Problems appear as amber warnings on the result.

3 Docker, explained from zero

Docker is why you could run this whole app with one click, without installing Python packages, Node.js, nginx or Tesseract on your Mac.

The problem Docker solves

To run the backend you'd normally need the right Python version, about 20 Python packages, and the Tesseract OCR program installed on your Mac, in versions that don't clash with your other projects. Then the same again for the frontend with Node.js. On a server, you'd do it all over again. Docker packages each part *together with everything it needs*, so it runs the same everywhere.

Four words you need

Image	A frozen snapshot of a small Linux computer with your app and everything it needs already installed. Built from a recipe called a **Dockerfile**. Think of it as an installer.
Container	A running copy of an image: an isolated mini-computer on your Mac. You can start, stop, delete and recreate it in seconds. Think of it as the running program.
Volume	Storage that lives *outside* containers, so data survives when a container is deleted or rebuilt. Your check history lives in a volume called `misinfo-data`.
Compose	`docker-compose.yml` describes several containers and how they connect, so one command starts the whole app.

What happened when you clicked "Run All Services"

VS Code ran `docker compose -f docker-compose.yml up -d --build`. Here's what Docker did:

1. **Read `docker-compose.yml`**: two services to run, *backend* and *frontend*.
2. **Built the backend image** from `backend/Dockerfile`. It downloaded a ready-made Python 3.12 Linux image, installed Tesseract, installed the Python packages, and copied in the `app/` code. That's the "Image misinfo-ai-v2-backend Built" line (about 2 minutes the first time).
3. **Built the frontend image** in two stages: Node.js converted the React code into plain HTML/JS/CSS files, then only those files were copied into a tiny nginx web server image.
4. **Created a private network** (`misinfo-ai-v2_default`) so the containers can reach each other by name. That's how nginx can forward to `http://backend:8000`.
5. **Created the volume** `misinfo-data` for the database.
6. **Started the backend**, then waited until its health check passed ("Healthy").
7. **Started the frontend** and connected your Mac's port 8080 to it. That's why `http://localhost:8080` works.

Why the second build is much faster

Docker caches each step. If you only change Python code, it reuses the installed packages and just copies the new code: seconds instead of minutes. The Dockerfiles copy the requirements files *before* the code for exactly this reason.

Ports, explained

A port is a numbered door on a computer. `"8080:8080"` in docker-compose.yml means "door 8080 on your Mac leads to door 8080 in the frontend container". The backend's door (8000) is deliberately *not* opened to your Mac: the only way in is through nginx. Fewer open doors means fewer things that can go wrong.

Everyday Docker commands

Run these in Terminal from the project folder (`cd ~/Downloads/misinfo-ai-v2`), or use the Docker Desktop app instead.

```
docker compose up -d --build      # build if needed, start everything in the background
docker compose ps                 # what's running and is it healthy?
docker compose logs -f backend    # watch the backend's log live (Ctrl+C to stop watching)
docker compose restart backend    # restart after editing backend/.env
docker compose down               # stop and remove containers (history is kept in the
volume)
docker compose down -v            # ...and ALSO delete the volume = wipe all history
```

After changing code or .env

A container runs the image it was built from, so code changes need a rebuild: `docker compose up -d --build` (or click **Run All Services** again). Changes to `backend/.env` only need a restart, because the file is read when the container starts.

Docker Desktop, the app

Containers lists the *misinfo-ai-v2* group. Use ▶ / ■ to start and stop, and click *backend* to see its live **Logs**; that's the first place to look when something goes wrong. **Images** shows the built images and **Volumes** shows *misinfo-data*. Docker Desktop must be running (whale icon in the menu bar) for any of this to work.

4 Running the app (four ways)

Use way A or B day to day. Use C when you're changing code and want instant reloads, and D when you want to pause the code and inspect it.

A. One click in VS Code (what we set up)

1. Open **Docker Desktop** and wait for "Engine running".
2. In VS Code, open `misinfo-ai.code-workspace` (File → Open Workspace from File).
3. Open `docker-compose.yml` and click the small ▷ **Run All Services** link above `services:`. Or go to Run and Debug and choose **"Run app in Docker"**.
4. When the terminal shows both containers *Started*, open **http://localhost:8080**.

B. From Terminal

```
cd ~/Downloads/misinfo-ai-v2
docker compose up -d --build
open http://localhost:8080
```

C. Development mode, without Docker

The backend and frontend run directly on your Mac, and both reload automatically when you save a file. You need Python 3.11+, Node 20+ and (for screenshots) `brew install tesseract`.

```
# Terminal 1: backend  →  http://localhost:8000/api/docs
cd backend
python3 -m venv .venv && source .venv/bin/activate     # first time only
pip install -r requirements-dev.txt -r requirements-ml.txt  # first time only
uvicorn app.main:app --reload

# Terminal 2: frontend  →  http://localhost:5173
cd frontend
npm install                                             # first time only
npm run dev
```

In VS Code the "first time only" lines are saved as tasks: *Terminal → Run Task → Setup: backend* and *Setup: frontend*. Vite forwards `/api` to port 8000, just as nginx does in Docker.

D. Debugging with breakpoints

After running the two setup tasks once, open **Run and Debug** (⇧⌘D), choose **"Full app (backend + frontend)"** and press **F5**. Click in the margin next to a line number (for example inside `aggregate()` in `aggregate.py`) to set a red-dot breakpoint. Run a check, and VS Code pauses on that line so you can inspect every variable. **"Backend: tests (debug)"** does the same inside the tests.

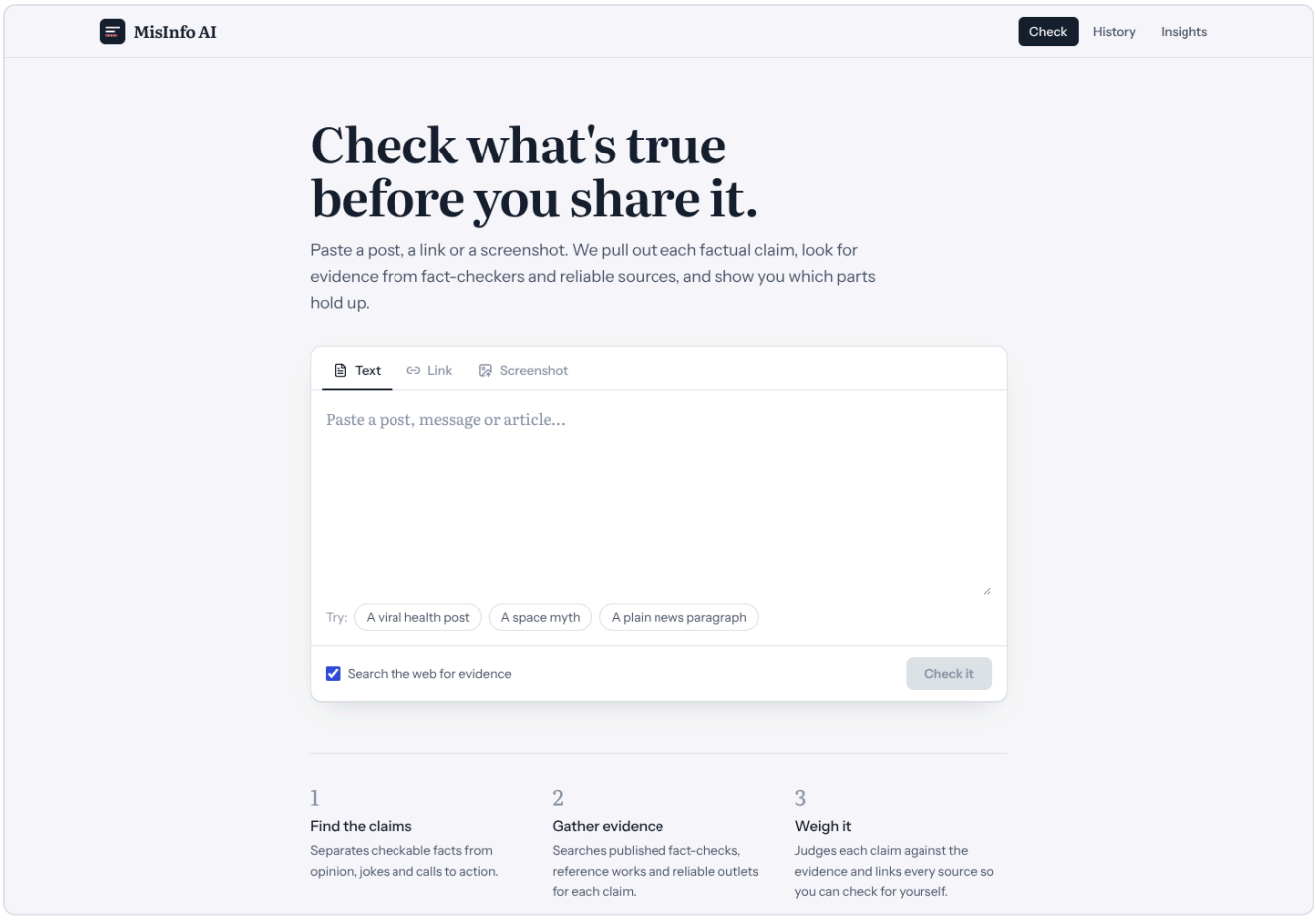
The interactive API docs

In mode C, `http://localhost:8000/api/docs` lists every endpoint with a **Try it out** button. It's the easiest way to see exactly what the backend returns, with no website involved. FastAPI generates it automatically from `schemas.py`.

The VS Code extensions we installed

Extension	What it gives you
Python, Pylance, Python Debugger	Autocomplete, error checking and breakpoints for the backend
Ruff	Finds Python mistakes and formats code when you save
ESLint, Tailwind CSS IntelliSense	Finds React mistakes; autocompletes Tailwind class names
Docker / Container Tools	The ▷ Run All Services links; containers and logs in the sidebar
DotENV, GitHub Actions, GitLens	Highlights .env files; shows CI results; shows who changed which line and when

5 Using the website and reading a result

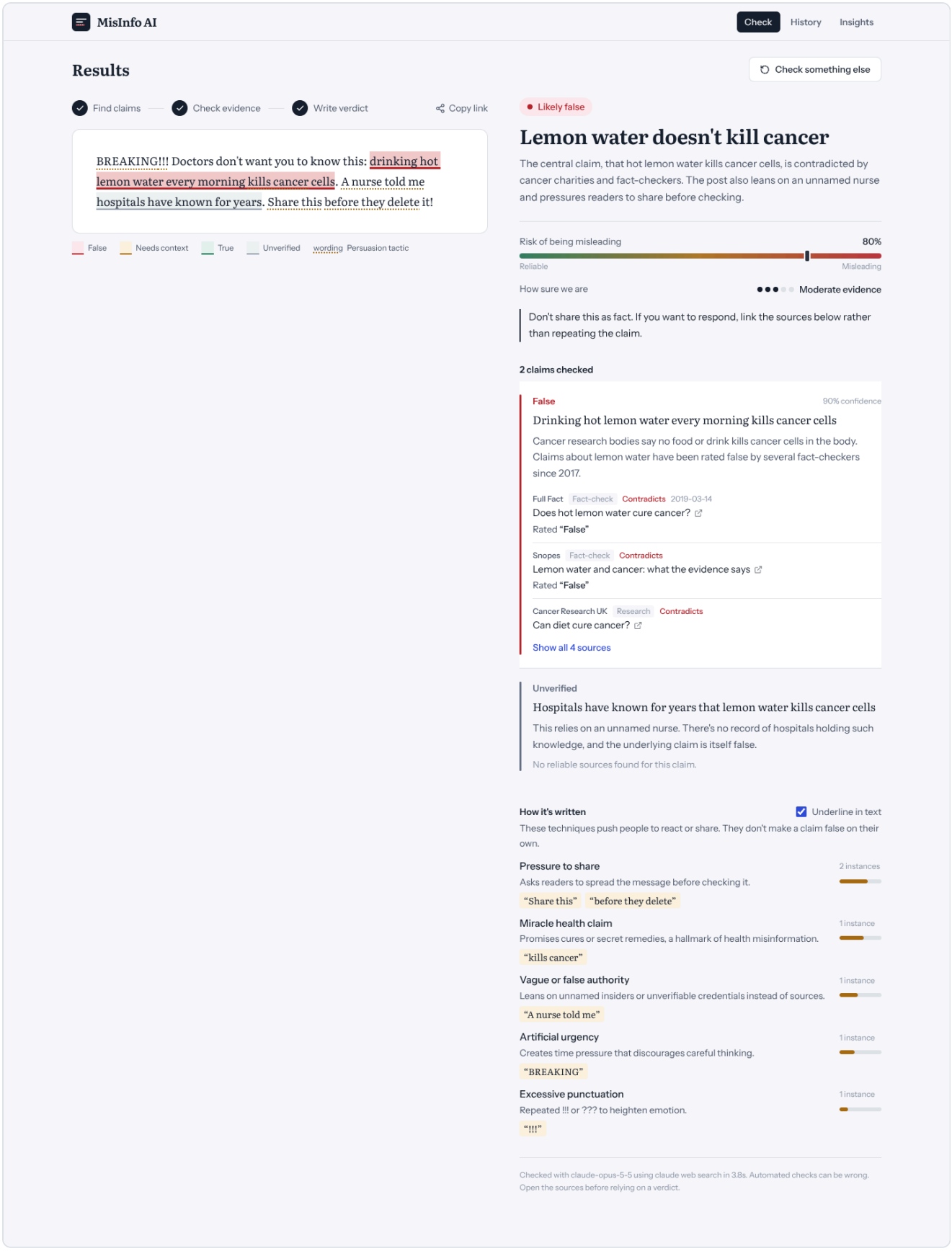


The Check page. Tabs switch between Text, Link and Screenshot (you can also paste an image anywhere with ⌘V). “Try” fills in an example. Untick “Search the web for evidence” for an offline-only check.

The pages

Page	Address	What it's for
Check	`/`	Submit something and watch it being checked
Saved result	`/a/<id>`	Any past check. **Copy link** on a result gives you this address
History	`/history`	Every past check, newest first. Click to reopen; bin icon to delete
Insights	`/insights`	Checks per day, verdict breakdowns, most common persuasion tactics

Reading a result



A finished check (example data from testing). Left: your text with claims coloured by verdict and tactics underlined. Right: the overall verdict, then each claim with its reasoning and sources.

Claim verdicts

Shown as	Means
False	Reliable evidence contradicts it, or fact-checkers rated it false
Needs context	Partly true, exaggerated, outdated, or reliable sources disagree
True	Reliable evidence confirms it as stated
Unverified	Not enough reliable evidence either way. The app prefers this to guessing

Overall verdicts

Shown as	When
Likely false	An important claim is false with at least 60% confidence
Needs context	Some claim is false (with weaker evidence) or needs context
Looks reliable	Every claim that could be checked is true, and enough of the text is covered
Unverified	Couldn't confirm or rule out the main claims
Nothing to check	Opinion or commentary with no factual claims

The two numbers

Risk of being misleading (the coloured bar, 0–100%) estimates how likely the content is to mislead. **How sure we are** (the dots) is *separate*: it says how much of the content is backed by evidence. That's why the lemon-water example in basic mode showed 54% risk but “Very little evidence”: the wording is suspicious, but nothing was actually verified.

Persuasion tactics

Dotted amber underlines mark techniques that push people to react or share. They are not proof of falsehood: true posts can say “BREAKING”. Others detected: miracle health claims (“secret cure”), emotional language (“shocking”), absolute certainty, scapegoating, false dilemmas, ALL CAPS and “!!!”. The full list lives in `backend/app/pipeline/tactics.py`.

Tactic	Example phrases
Pressure to share	“share before they delete it”, “forward to everyone”
Artificial urgency	“BREAKING”, “act now”, “before it's too late”
Conspiracy framing	“they don't want you to know”, “wake up”, “cover-up”
Vague or false authority	“a nurse told me”, “insiders say”, “studies show”

6 How the verdict is calculated

All of this is in `backend/app/pipeline/aggregate.py`. It has no machine learning, just about 40 lines of transparent rules, so you can explain every number.

Step 1: weigh each claim

Each claim has two numbers: **checkworthiness** (how important it is, from the AI) and **confidence** (how strong its evidence is). Its influence is $importance \times confidence$. Unverified claims count for nothing either way.

Step 2: coverage

Coverage is the share of the content that's backed by evidence, from 0 to 1. With lots of strong evidence on important claims, coverage is high.

Step 3: risk

```
risk = coverage × (what the evidence says) + (1 - coverage) × (starting guess from the wording)

evidence says:  False = 1.0   Needs context = 0.55   True = 0.0
starting guess: 0.2 + 0.4 × tactics score  (± a little from the style model), kept between 0.03 and 0.85
```

So when evidence is strong, the evidence dominates. When there's none, the wording guess fills in, but it can never go above 0.85 or produce a confident verdict.

Step 4: the label and confidence

- **Likely false** if any important claim (checkworthiness ≥ 0.4) is False with confidence ≥ 0.6. Risk is raised to at least 70%.
- **Needs context** if any claim is False or Needs context. Risk is at least 40%.
- **Looks reliable** if all checkable claims are True and coverage ≥ 45%.
- Otherwise **Unverified**.
- **Confidence** = 0.1 + 0.85 × coverage, capped at 95%. It can never be 100%.

> **Worked example**
>
> Two important claims (checkworthiness 0.9 each). Claim A is `False` at 90% confidence; claim B is `Unverified`. The wording has lots of tactics (score 0.9), so the starting guess is 0.2 + 0.4 × 0.9 = 0.56.
>
> coverage = (0.9 × 0.9) ÷ (0.9 + 0.9) = 0.45 · risk = 0.45 × 1.0 + 0.55 × 0.56 = 0.76 · confidence = 0.1 + 0.85 × 0.45 = 0.48.
> A is important and strongly false, so the result is **Likely false, 76% risk, moderate evidence**.

How a single claim gets its verdict

With Claude, the claim, the original text and the numbered evidence go to the model with instructions to act as a careful, neutral fact-checker (the prompt is `VERIFY_SYSTEM` in `llm/base.py`). It may run up to 3 web searches, then must return a structured answer: a verdict, a confidence, a reason, and which evidence supports or contradicts the claim. **Without an AI**, the app looks for a published fact-check of a very similar claim and uses its rating (“False”, “Half True”…). If none matches, the claim stays Unverified.

7 Settings and API keys

Every setting lives in `backend/.env`, a plain text file of `NAME=value` lines. `backend/.env.example` lists them all with explanations. After editing, restart the backend.

> **Keep .env private**
>
> It holds your API keys. It's listed in `.gitignore`, so Git never uploads it. Never paste keys into code, screenshots or chats.

The settings that matter most

Setting	Default	What it does
`GOOGLE_FACTCHECK_API_KEY`	empty	Turns on Google Fact Check search. Free. The most valuable single key.
`ANTHROPIC_API_KEY`	empty	Turns on Claude. Without it the app runs in basic mode.
`LLM_PROVIDER`	`anthropic`	`anthropic` , `ollama` (free local model) or `none`
`LLM_MODEL`	`claude-opus-5-5`	The careful model that judges claims
`LLM_FAST_MODEL`	`claude-haiku-5-5`	The quick, cheap model that finds claims and writes summaries
`LLM_WEB_SEARCH`	`true`	Lets Claude search the web while judging (billed per search)
`ENABLE_WIKIPEDIA`	`true`	Wikipedia evidence (free)
`BRAVE_API_KEY` / `TAVILY_API_KEY`	empty	Optional extra web search
`RATE_LIMIT`	`30/minute`	Checks allowed per visitor; protects your API bill once deployed
`STORE_INPUT_TEXT`	`true`	`false` = history keeps verdicts but not what people pasted
`MAX_CLAIMS`	`5`	Most claims checked per request (more = slower and costlier)
`DATABASE_URL`	empty	Empty = SQLite file. Set a Postgres URL when deploying

Where each key comes from

Key	Where	Cost
Google Fact Check	console.cloud.google.com → new project → enable “Fact Check Tools API” → Credentials → Create API key	Free
Anthropic	console.anthropic.com → API Keys	Pay per use; a check costs a few pence
Brave Search	api-dashboard.search.brave.com	Free tier available
Tavily	tavily.com	Free tier available

How settings reach the code: `config.py` defines every setting with a default; the `pydantic-settings` library reads environment variables and `.env`, matching names case-insensitively. In Docker, `docker-compose.yml` passes `backend/.env` into the backend container.

How we got your Google Fact Check key

This is exactly what was done for your key, in your own Chrome while signed in to your Google account. Google Cloud is Google's site for developer tools; a **project** is a folder that groups APIs and keys, and an **API key** is a password-like string that lets the app call an API and tells Google who is calling.

1. Opened `console.cloud.google.com` and created a **new project** called **MisInfo AI** (ID `misinfo-ai-513217`). A separate project keeps this key apart from your other work, such as `hotel-mgt-test`.
2. Went to **APIs & Services → Library**, searched for **Fact Check Tools API** and clicked **Enable**. An API has to be switched on in a project before any key can use it.
3. Went to **APIs & Services → Credentials → Create credentials → API key** and named it **misinfo-ai backend**.
4. Under **API restrictions**, chose **Restrict key** and ticked only **Fact Check Tools API**. If the key ever leaked, it couldn't be used for any other Google service.
5. Left **Application restrictions** on **None**. The key is used by the backend server, not a web page, so website and app restrictions don't apply. (Once deployed, you could restrict it to your server's IP address.)
6. Clicked **Create**, and Google showed the key once in a dialog.
7. You copied the key into `backend/.env` on the line `GOOGLE_FACTCHECK_API_KEY=` and saved. You pasted it yourself because, for safety, Claude's remote tools aren't allowed to write `.env` files on your Mac.
8. Rebuilt and restarted the containers (Run All Services). Docker reads `.env` only when it *creates* a container, so a running one never sees a new key until it's recreated.

Cost: free. The Fact Check Tools API needs no billing account. **To check it works:** open `localhost:8080/api/health`; `"google_factcheck": true` means the key was picked up. **To replace or delete it:** Google Cloud → project MisInfo AI → APIs & Services → Credentials → *misinfo-ai backend*. There you can regenerate the key (then paste the new one into .env) or delete it. Do this straight away if the key ever appears in a screenshot, chat or commit.

8 Backend code tour, file by file

Everything is in `backend/app/`. Every file now opens with a comment explaining its job, so this is the map. Open the file for the detail.

The core

File	Job
`main.py`	The web server. Defines every URL (`/api/health`, `/api/analyze/stream`, `/api/history`…), validates requests, applies rate limits and CORS, and streams results. **Start reading here.**
`config.py`	All settings with their defaults (section 7)
`schemas.py`	The shape of every piece of data sent or received: `AnalysisResult`, `ClaimResult`, `Evidence`, `Tactic`… The frontend depends on these exact field names.
`db.py`	Saves and loads history (SQLite via SQLModel). One table, `analyses`; each row keeps a summary plus the full result as JSON.

`pipeline/`: the checking steps

File	Job
`orchestrator.py`	The conductor. Runs every step in order, checks claims in parallel, enforces the 75-second budget, falls back when something fails, and yields the streaming events.
`ingest.py`	Turns links into article text (trafilatura) and screenshots into text (Tesseract OCR). Blocks links to private network addresses (SSRF protection).
`retrieval.py`	Evidence search: Google Fact Check, Wikipedia, Brave, Tavily, all at once. To add a new source, add a function here.
`tactics.py`	Persuasion-tactic detection with word-boundary regular expressions. To add a tactic, add a `TacticDef`.
`style.py`	Loads and runs the optional trained model from `backend/models/style/`
`aggregate.py`	The verdict maths (section 6), plus the default headlines and guidance text

`llm/`: the AI layer

Three interchangeable “providers” all have the same three methods (`extract_claims`, `verify_claim`, `summarize`), so the rest of the app doesn't care which one is active.

File	Job
`__init__.py`	`get_provider()` picks the provider from `LLM_PROVIDER`. If Claude has no key, it falls back to the no-AI provider.

`base.py`	The prompts, the JSON answer formats (schemas), and the provider interface. **Edit the prompts here** to change how the AI behaves.
`anthropic_provider.py`	Claude: Haiku finds claims and writes summaries; Opus judges claims with optional web search. Uses “forced tools” so answers always come back as valid JSON.
`ollama_provider.py`	A free local model via Ollama. Private, but slower and weaker, with no web search.
`heuristic_provider.py`	No AI: sentence rules to find claims, fact-check matching to judge them. Never guesses.

`services/`: shared helpers

File	Job
`http.py`	One shared HTTP client with a 6-hour disk cache, so repeat lookups are instant and save API quota
`sources.py`	Website reputation: is a link a fact-checker, news outlet, government body, journal or unknown blog, and its starting trust score. Also flags satire sites.

Other backend files

`tests/`	21 automated tests (section 11)
`requirements*.txt`	Python packages: main app, optional ML extras, dev tools
`Dockerfile`	The backend image recipe (commented line by line)
`models/style/`	Where a trained style model goes (empty by default)
`data/`	Created at run time: `misinfo.db` and `cache/`. Inside the Docker volume when running in Docker.

9 Frontend code tour, file by file

Everything is in `frontend/src/`. React builds the page from **components**, small functions that each draw one part of the screen and redraw automatically when their data changes.

How data flows on the Check page

```
InputPanel  --(you click "Check it")-->  useAnalysis().run()
                                                |
                                  api.analyzeStream()  --POST-->  backend
                                                |   <--- events stream back
                                  onEvent() updates the state
                                                |
ResultView  <--(reads the state)---------------+
   |-- AnnotatedText   your text with highlights
   |-- VerdictPanel    verdict, headline, risk, confidence
   |-- ClaimNotes      each claim + reasoning + sources
   +-- WritingSignals  persuasion tactics + style score
```

File	Job
`main.jsx`	Entry point: loads fonts and styles, starts React
`App.jsx`	Header, navigation, page routing (`/`, `/a/:id`, `/history`, `/insights`), and the basic-mode / server-down banners
`lib/api.js`	Every call to the backend, including reading the live event stream by hand (the browser's built-in tool can't send POST requests)
`lib/useAnalysis.js`	The Check page's “brain”: holds the current check's state and updates it as each event arrives
`lib/verdicts.js`	Labels and colours for each verdict. Change wording here and it changes everywhere.
`components/InputPanel.jsx`	The Text / Link / Screenshot box, drag-and-drop, paste-an-image, example buttons
`components/ResultView.jsx`	The two-column result layout, the Copy link button and the colour legend
`components/AnnotatedText.jsx`	Your text with claim highlights and tactic underlines, handling overlapping spans
`components/Verdict.jsx`	Progress steps, verdict badge, headline, risk bar, confidence dots, guidance, warnings
`components/ClaimNotes.jsx`	One note per claim: verdict, reasoning, sources (with fact-checkers' ratings)
`components/WritingSignals.jsx`	The “How it's written” panel

`pages/*.jsx`	Check, Report (saved result), History, Insights (charts with Recharts)
`index.css`	Claim highlight colours, tactic underlines, loading shimmer, the verdict's entrance animation
`tailwind.config.js`	The colour palette and fonts. Tailwind lets you style with short class names like `text-sm text-ink`.
`vite.config.js`	Dev server settings, including forwarding `/api` to the backend on port 8000
`nginx.conf` , `Dockerfile`	How the built site is served in Docker (commented line by line)

> **Changing the website**
>
> Run it in development mode (section 4C) and edit any file in `src/`: the browser updates the moment you save. For Docker, rebuild afterwards.

10 Training the style model

This is where your datasets come in. It's optional: it adds a "resembles known misinformation" score, and it's how you get honest ML results to report. Full instructions are in `training/README.md`.

The three steps

```
cd training
pip install pandas pyarrow scikit-learn joblib

# 1. Clean, de-duplicate and split your raw datasets
python prepare.py --raw /path/to/your/raw --out data

# 2. Train the quick baseline (minutes on a laptop); saves straight into the app
python baseline.py --data data --out ../backend/models/style

# 3. (Optional, needs a GPU, e.g. free Google Colab) fine-tune a transformer
python finetune.py --data data --model microsoft/deberta-v3-base --out
../backend/models/style
```

Then rebuild the backend (`docker compose up -d --build`) and the start-up log shows `style=TF-IDF baseline`.

What each script does

File	Job
`loaders.py`	Reads each dataset's own file format with **corrected labels** (section 13)
`prepare.py`	Removes URLs and @handles, drops exact and near-duplicate texts, and splits into train / val / calib / test so that no event or story appears in two splits
`baseline.py`	TF-IDF + logistic regression, calibrated. Reports per-dataset and leave-one-dataset-out results.
`finetune.py`	Fine-tunes a transformer and exports it to ONNX, so the server runs it without PyTorch
`evaluation.py`	Shared metrics: ROC-AUC, F1, calibration, threshold choice, reports

The number to trust

Each run writes a `REPORT.md`. The most honest figure is **leave-one-dataset-out (LODO) AUC**: train without dataset X, then test on X. It shows whether the model learned something general or just each dataset's quirks. Expect it to be clearly lower; that gap is why v2 lets evidence decide verdicts.

Raw folder layout: the same folder names v1 used: `liar/` (or `lair/`), `twitter15/`, `twitter16/`, `pheme/`, `fakenewsnet/`, `CoAID-master/`, `FakeHealth/`. Your iCloud `ml/data/raw` folder should work as it is.

11 Tests, CI and Git

Automated tests

```
cd backend  && pytest -q          # 21 tests: tactics, claims, verdict maths, full API,
failure handling
cd training && pytest -q tests    # label fixes + prepare + baseline on synthetic data
cd frontend && npx eslint src && npm run build
```

The tests need no API keys and no internet: a fake AI provider stands in for Claude, and outside APIs are simulated. Several tests lock in v1 bug fixes so they can't come back. In VS Code, the **Testing** panel (flask icon) runs them with one click.

CI: automatic checks on GitHub

`.github/workflows/ci.yml` runs every check above, plus a Docker build, each time you push. Results are in your repo's **Actions** tab: a green tick means everything passed. Your first push of v2 passed in under 2 minutes.

Git branches

`main` still holds the original dissertation code; `v2` holds this version. When you're happy, merge v2 into main through a pull request on GitHub. The everyday loop:

```
git status                        # what changed?
git add -A && git commit -m "..."  # save a snapshot with a message
git push                          # upload to GitHub (CI runs automatically)
```

> **Your Downloads copy isn't a Git repository**
>
> `~/Downloads/misinfo-ai-v2` came from a zip, so it has no Git history. For real development, once your network allows it: `git clone https://github.com/pakson556/misinfo-ai.git`, then `git checkout v2`, and copy your `backend/.env` across.

12 Troubleshooting

You see	Cause	Fix
“failed to connect to the docker API … docker.sock”	Docker Desktop isn't running	Open Docker Desktop, wait for “Engine running”, try again
“failed to fetch oauth token … i/o timeout”, or `git clone` “Couldn't connect to server”	Your network blocks non-browser traffic (some university and public Wi-Fi does)	Use home Wi-Fi or a phone hotspot for the first build. After that, Docker works offline from its cache.
“port is already allocated” for 8080	Something else uses port 8080	Stop the other app, or change `"8080:8080"` to `"8081:8080"` in docker-compose.yml and use localhost:8081
Yellow “Running in basic mode” banner	No Anthropic key	Add `ANTHROPIC_API_KEY` to backend/.env and restart the backend
Red “analysis server isn't responding” banner	Backend stopped or crashed	Docker Desktop → backend → Logs. The last lines say why.
Most claims “Unverified”	No evidence sources or AI configured	Add the Google Fact Check key (free) and an Anthropic key
Amber “google_factcheck unavailable (HTTPStatusError)”	Wrong key, or the API isn't enabled on the Google project	Check the key in .env; enable “Fact Check Tools API” in Google Cloud
Health shows `"google_factcheck": false` after adding the key	The container was created before the key was saved	Save .env, then Run All Services again so the backend container is recreated
Backend “unhealthy”; log shows `PermissionError … /app/app/main.py`	Code files on your Mac were saved as owner-only, and the container runs as a limited user	Fixed in the Dockerfile (`chmod -R a+rX`). Rebuild with Run All Services.
“No readable text found in the image”	Blurry or tiny screenshot	Use a sharper, larger screenshot
Code changes don't appear	Containers still run the old image	Click Run All Services again, or `docker compose up -d --build`
“Too many checks in a short time”	Rate limit reached	Wait a minute, or raise `RATE_LIMIT` in .env

The golden rule

When anything goes wrong, read the **backend log** first: Docker Desktop → Containers → misinfo-ai-v2 → backend → Logs, or `docker compose logs -f backend`. The first line after start-up shows exactly which AI model and evidence sources are switched on.

13 What changed from v1, and why

The full audit is in `docs/V1_AUDIT.md`. These are the points worth knowing, for your own understanding and for interviews.

v1 problem	Effect	v2
The "BERT" model was a *sentiment* model	Cheerful text scored as riskier	Removed. Claims are judged against evidence.
Twitter15/16 "non-rumor" labelled as misinformation	Ordinary tweets taught as false	Correct mapping; unverified tweets dropped
CoAID label column didn't exist	All fake COVID news labelled real	Label taken from the file name
FakeHealth ratings misread and mismatched to articles	Wrong labels	Stars < 3 = misinformation, joined by news id
Keyword-labelled dataset (Brazil anti-vax)	The model learned the keywords back; inflated accuracy	Excluded
Random split, no de-duplication	Near-copies in train and test inflated scores	De-duplication + group-aware splits + LODO evaluation
Saved threshold never used; hand-tuned multipliers	The "probability" wasn't a probability	Transparent, documented verdict maths
Hardcoded answers for demo examples	Looked good on demos, failed elsewhere	No topic-specific code
Up to 6 minutes per request, sequential calls	Frontend timed out at 30 s	Parallel, cached, 75 s budget, live streaming
Hardcoded localhost, no Docker or tests	Couldn't be deployed	Env config, Docker, 23 tests, CI

Your cleaner v1 run scored **F1 0.63 / ROC-AUC 0.84**. The 0.96 accuracy came after adding keyword-labelled data, which made the task easier without making the model better. Being able to explain this is a strength, not a weakness.

14 Glossary

API	A set of URLs a program calls to get data or ask for work. The backend is an API; so are Google Fact Check and Claude.
API key	A secret password that identifies you to a paid or rate-limited API
Async	Code that can wait for the network without freezing, so the server handles many requests at once
Backend / frontend	The server-side program (Python) / the part in the browser (React)
Cache	Saved copies of answers so repeat requests are instant
CI	Continuous Integration: automatic tests on every push (GitHub Actions)
Claim	One checkable factual statement pulled from the input
Container / image	A running isolated app / its frozen template (section 3)
CORS	Browser security rule about which websites may call an API
Endpoint / route	One URL of an API, e.g. `/api/history`
Environment variable	A named setting given to a program from outside its code (`.env`)
FastAPI	The Python framework the backend is built with
JSON	The text format used to exchange data: `{"verdict": "REFUTED"}`
LLM	Large language model, e.g. Claude
nginx	A fast web server; serves the website and forwards API calls in Docker
OCR	Reading text from an image (Tesseract)
ONNX	A portable format for trained models; runs fast on CPU without PyTorch
Port	A numbered door on a computer that a program listens on (8080, 8000)
Prompt	The instructions given to an AI model
Pydantic	The library that defines and validates data shapes (`schemas.py`)
Rate limit	Maximum requests per visitor per minute
React	The library the website is built with, out of components
Regex	Regular expression: a pattern for finding text
SQLite	A database stored as a single file
SSE	Server-Sent Events: a response that delivers updates over time
SSRF	An attack that tricks a server into fetching internal addresses; blocked in `ingest.py`
Tailwind	CSS styling with short utility class names
Vite	The frontend's dev server and build tool
Volume	Docker storage that survives container rebuilds